

TTP - Time-Triggered Protocol for Fault-Tolerant Real-Time Systems

Milestone 1: Technical Understanding and Research Framing

Jaleel Ur Rehman
Electronics Engineering
Hochschule Hamm-Lippstadt
jaleel.ur-rehman@stud.hshl.de

Problem Statement and Motivation

- Distributed real-time control systems must satisfy two concurrent constraints simultaneously:
 - Timely behavior:** hard deadlines on computation and communication must be guaranteed.
 - Dependable behavior:** the system must continue to operate correctly in the presence of hardware faults, communication failures, or external disturbances.
- Satisfying both at the same time is especially difficult when active fault tolerance requires hardware and software redundancy.
- Early 1990s protocols fell into two categories:
 - Event-triggered** (CAN, J1850): bandwidth allocated dynamically on demand. Worst-case timing is load-dependent and hard to bound. Complex schedulability analysis. Priority inversion possible during fault handling.
 - Time-triggered:** communication is periodic and synchronized to a global clock, but no single protocol at the time integrated all required fault-tolerance services.
- No existing 1993-era protocol combined clock synchronization, membership service, redundancy management, and guaranteed message delivery in one protocol without adding extra messages.

Core Technical Contribution

- **Static TDMA schedule:** Every node's send and receive slot is fixed at design time and driven solely by a globally synchronized time base. No runtime arbitration.
- **Sparse time base:** All send/receive operations are aligned to a discrete time lattice with granule $g \approx 2$ bits at 250 kbit/s. Drastically reduces the number of states that must be modeled or analyzed.

- **Controller State (C-state):** A three-field shared state {mode, time, membership} implicitly agreed upon via CRC that covers both data and C-state. A frame is only accepted if the CRC matches, meaning sender and receiver share the same C-state.
- **Integrated services zero extra frames:** Clock synchronization, membership tracking, and redundancy management are all achieved through existing TDMA timing and C-state – no additional protocol messages.

Central Algorithm and Architecture

TDMA Scheduling

- The TDMA round is divided into FTU slots; each FTU slot holds 1 or 2 frame slots depending on the fault-tolerance class.
- The schedule, message formats, and task dispatcher are pre-defined per operational mode.
- Bus access is entirely governed by global time – no contention, no arbitration overhead.

C-State Agreement

- A CRC mismatch means either the data are corrupted *or* sender/receiver C-states differ. Either way the frame is rejected.
- Valid frame acceptance $\Leftrightarrow$ sender and receiver share the same {mode, time, membership}.
- This provides group acknowledgment and C-state consistency checking without any extra messages.

Clock Synchronization

- Each receiver knows in advance exactly when a frame should arrive (from the TDMA schedule).
- The difference between expected and actual arrival time gives an estimate of the sender-receiver clock offset.
- A fault-tolerant averaging algorithm is applied periodically to correct all local clocks toward a common global time.

Membership Service

- Each FTU's membership bit is set to 1 when it transmits a valid frame and to 0 when its slot passes without a valid received frame.
- A node that discards more frames than it accepts in one TDMA round detects C-state divergence and exits the active ensemble to prevent clique formation.

Fail-Silent Enforcement

- Nodes either produce correct output or *no* output at all.
- Enforced by hardware-level space/time redundancy and error-detection logic that shuts down the node before any erroneous output is transmitted.

Four Fault-Tolerant Configuration Classes

Table 1: TTP Fault-Tolerant Configuration Classes

Tolerance of	I	II	III	IV
Perm. node failure	0	1	1	2
Perm. comm. failure	1	1	1	1
Trans. node failure	0	$1/R_{c_{int}}$	$1/R_{c_{int}}$	$1/TR$
Trans. comm. failure	1 of 2	1 of 2	3 of 4	3 of 4

- **Class I:** 1 node per FTU, 2 frames per FTU. Simplest configuration; suited when cabling failures dominate.
- **Class II:** 1 active node per FTU, tolerates 1 permanent node failure and transient node failures within the reintegration cycle.
- **Class III:** Like Class II but with 4-channel communication redundancy (3 of 4 frames must be correct).
- **Class IV:** 2 active + 1 shadow node per FTU, 4 frames per FTU. Most robust; tolerates up to 2 permanent node failures. Shadow nodes maintain warm backup state and immediately take over an unacknowledged slot.

Assumptions and Scope Limitations

- Nodes are **fail-silent**: Byzantine faults (inconsistent outputs to different receivers) are *outside* the assumed failure model.
- Channel failures are modeled as **omission failures** only (no babbling-idiot channel faults assumed beyond hardware-enforced fail-silence).
- System topology is **static and fully known at design time**; adding or removing nodes requires full schedule redeployment.
- All communication is assumed to be **periodic**; aperiodic traffic is accommodated only via pre-defined mode changes.
- External time synchronization is out of scope.
- The membership and mode-change protocols assume **no node fails at the exact instant of a configuration change** – an assumption that cannot be guaranteed under fully asynchronous fault models.

Table 2: Assumed Failure Rates in TTP

Failure Type	Rate (h^{-1})
Permanent node failure	10^{-3}
Permanent channel failure	10^{-4}
Transient node failure	10^{-5}
Transient channel failure	10^{-6}

References

- [1] H. Kopetz and G. Grünsteidl, “TTP – A Time-Triggered Protocol for Fault-Tolerant Real-Time Systems,” in *Proc. 23rd Int. Symp. Fault-Tolerant Computing (FTCS-23)*, pp. 524–533, Toulouse, France, June 1993.
- [2] H. Kopetz, “Event-Triggered versus Time-Triggered Real-Time Systems,” in *Proc. Int. Workshop on Operating Systems of the 90s and Beyond*, LNCS vol. 563, pp. 87–101, Springer-Verlag, 1991.
- [3] H. Kopetz and W. Ochsenreiter, “Clock Synchronization in Distributed Real-Time Systems,” *IEEE Trans. Computers*, vol. 36, no. 8, pp. 933–940, Aug. 1987.
- [4] SAE Handbook, Vol. 2. Society of Automotive Engineers, PA, USA, 1992.